

PlatePilot Cost Model & Zero-Capital Infrastructure Strategy

AWS + AI + Billing Cost Analysis with Free Alternatives

Version 1.0

Objective: Define a realistic and financially sustainable infrastructure strategy for PlatePilot, starting with **\$0 capital** and scaling progressively as revenue grows.

Core Principle: Build the product with free or near-free tools initially, then migrate to enterprise-grade infrastructure only when user traction and recurring revenue justify the cost.

1. Executive Summary

Because you are starting with **no initial capital**, the correct strategy is:

Use free tiers and open-source alternatives for every non-essential paid service, while designing the architecture so migration to premium services is straightforward.

This approach allows you to:

- Launch the MVP at virtually no cost.
 - Validate product-market fit.
 - Acquire initial users.
 - Generate first revenues.
 - Upgrade infrastructure gradually.
-

2. Recommended Zero-Capital Technology Stack

Function	Enterprise Option	Free / Bootstrap Alternative
Backend Hosting	AWS App Runner	Render (free tier)
Database	Amazon RDS for PostgreSQL	Neon (free PostgreSQL)
Cache	Amazon ElastiCache for Redis	Upstash (free Redis)
File Storage	Amazon S3	Cloudinary or Cloudflare R2
Secrets Management	AWS Secrets Manager	Render environment variables
Monitoring	Amazon CloudWatch	Sentry free tier
Analytics	Custom + AWS	PostHog free tier
CI/CD	AWS CodePipeline	GitHub Actions
Push Notifications	Custom provider	Firebase Cloud Messaging (free)
Email	Amazon SES	Resend or Brevo free tier
AI	OpenAI	Rule-based engine first, then optional AI
Payments	Stripe	Same provider; no monthly fee, only transaction fees

3. Recommended Bootstrap Architecture (\$0–\$10/month)

Flutter Mobile App

↓ HTTPS

Spring Boot Backend on Render (Free)

↓

PostgreSQL on Neon (Free)

↓

Redis on Upstash (Free)

↓

Cloudinary / Cloudflare R2

↓

Firebase Cloud Messaging (Free)

↓

GitHub Actions (Free)

↓

PostHog + Sentry (Free)

4. Monthly Cost by Development Stage

Stage 0 — Development and MVP (0–100 users)

Service	Cost
Render	\$0
Neon PostgreSQL	\$0
Upstash Redis	\$0
Cloudinary	\$0
Firebase Cloud Messaging	\$0
GitHub Actions	\$0
PostHog	\$0
Sentry	\$0
Email Provider	\$0
Stripe	\$0 fixed cost
AI	\$0 (disabled initially)

Total Estimated Monthly Cost

\$0 – \$5/month

Stage 1 — Early Validation (100–1,000 users)

Service	Estimated Cost
Hosting	\$0 – \$25
Database	\$0 – \$19
Redis	\$0 – \$10

Storage	\$0 – \$10
Monitoring	\$0
Email	\$0 – \$15
AI	\$10 – \$50
Stripe	Variable transaction fees

Total Estimated Monthly Cost

\$10 – \$120/month

Stage 2 — Growth (1,000–10,000 users)

Service	Estimated Cost
Hosting	\$25 – \$100
Database	\$19 – \$100
Redis	\$10 – \$50
Storage	\$10 – \$50
Email	\$15 – \$50
Monitoring	\$0 – \$50
AI	\$50 – \$500

Total Estimated Monthly Cost

\$130 – \$900/month

Stage 3 — Scale (10,000–100,000 users)

Service	Estimated Cost
----------------	-----------------------

Infrastructure \$500 – \$5,000+

AI \$500 –
\$10,000+

Monitoring \$50 – \$500

Email \$50 – \$500

Total Estimated Monthly Cost

\$1,000 – \$15,000+/month

5. AI Cost Strategy

Recommended Approach

Phase 1 — No Paid AI

Use deterministic rule-based logic for:

- Meal filtering
- Budget calculations
- Pantry matching
- Grocery generation

Phase 2 — Limited AI Enrichment

Use OpenAI only for:

- Recipe descriptions
- Alternative meal suggestions
- Natural-language explanations

Phase 3 — Advanced Personalization

Expand AI usage after recurring revenue exists.

6. Why Rule-Based Logic First

The core recommendation problem is deterministic:

- Budget constraints
- Cooking time
- Pantry availability
- Dietary restrictions
- Household size

These are best handled by algorithms and business rules, not necessarily by LLMs.

This dramatically reduces costs while improving reliability.

7. Stripe Cost Model

Stripe has:

- No monthly subscription fee.
- Charges only when payments are processed.
- Fees vary by country and payment method.

Example Scenario

If you charge \$9.99/month:

- Gross revenue per user $\approx$ \$10
 - Stripe deducts transaction fees
 - Remaining revenue funds infrastructure and profit
-

8. Recommended Subscription Pricing

Plan	Monthly Price
Free	\$0
Premium	\$7.99 – \$9.99
Family	\$14.99 – \$19.99

9. Break-Even Analysis (Illustrative)

Assuming:

- Premium price = \$9.99/month
- Infrastructure costs = \$100/month

Approximate paying users needed to cover infrastructure:

Around 10–15 premium subscribers

This demonstrates the attractive economics of SaaS.

10. Migration Roadmap

Phase 1 — Bootstrap

- Render
- Neon
- Upstash
- Rule-based engine

Phase 2 — Paid Validation

- Render paid plan or initial AWS adoption
- Limited AI calls

Phase 3 — Production

- Amazon Web Services
- OpenAI integration
- Full observability

Phase 4 — Scale

- Amazon ECS
 - Multi-AZ architecture
 - Cost optimization
-

11. Recommended Startup Strategy

Given your current situation:

Do not spend money on infrastructure until users demonstrate clear value and willingness to pay.

Priority order:

1. Build MVP.
 2. Validate with real users.
 3. Iterate.
 4. Launch premium features.
 5. Reinvest early revenue into infrastructure.
-

12. Practical Recommendation for PlatePilot

Use the following stack immediately:

- Flutter frontend
 - Spring Boot backend
 - [Render](#)
 - [Neon](#)
 - [Upstash](#)
 - Firebase Cloud Messaging
 - [PostHog](#)
 - [Sentry](#)
 - Stripe
 - Rule-based recommendation engine first
-

13. Final Strategic Conclusion

PlatePilot can be launched with **virtually zero infrastructure cost** while retaining an architecture that is fully compatible with future migration to Amazon Web Services and paid AI services.

The most important principle is:

Spend money only when the product has demonstrated real user adoption and recurring revenue.

This approach minimizes risk, preserves capital, and allows you to focus on what matters most: building a product that users love and are willing to pay for.